Exam Programming for Physics: Physics-Informed Neural Networks to Solve Differential Equations

Vasco Verwimp

28 May 2026

Contents

Chapter 1

Introduction

Traditional approaches to solving differential equations rely on analytical methods or classical numerical techniques such as finite difference methods, finite element methods, and spectral methods. However, these methods often struggle with high-dimensional problems, complex geometries, and inverse problems. In recent years, machine learning has emerged as a powerful alternative for solving differential equations, offering flexibility and scalability that traditional methods cannot provide.

Physics-Informed Neural Networks (PINNs) combine the neural networks with physical constraints encoded in differential equations. This approach has proven effective for predicting system evolution, and can be extended to extracting the physical parameters underlying observations.

PINNS are applied to two classical differential equation problems:

1. **The Damped Harmonic Oscillator**: A second-order ordinary differential equation (ODE) representing mechanical vibrations with friction.

2. **Burgers' Equation**: A nonlinear partial differential equation (PDE) that combines advection and diffusion processes.

We investigate how PINNs can solve these equations by training neural networks to simultaneously satisfy the observed data and the underlying physics constraints. This will be used to extrapolate into a time-domain for which no observational data was given. As a benchmark, we will use a standard machine learning model: a simple neural network. Additionally, we will compare two different implementations: a fully blind PINN, which can only train on the training domain and has no information about how "good" it is doing in the extrapolation domain during its training, and an extended physics PINN, for which the physics loss will also be applied to the extrapolation domain so it can improve in this regard, even though no observational data is given.

1.1 Physics-Informed Neural Networks (PINNs)

A Physics-Informed Neural Network is a neural network that is trained to solve differential equations by incorporating the equation's physics directly into the loss function. The network learns to approximate the solution $u(x, t)$ such that it satisfies both:

1. The observed data (data loss, $\mathcal{L}_{\text{data}}$)

2. The differential equation (physics loss, $\mathcal{L}_{\text{phys}}$)

3. Initial and boundary conditions (constraint loss, $\mathcal{L}_{\text{ic}}$)

The key innovation that has greatly increased the ease of use of PINNs is automatic differentiation to compute derivatives. This allows us to directly enforce the physics constraints encoded in differential equations.

Let $u(x,t)$ denote the solution to a differential equation. A neural network $u_\theta(x,t)$ is trained to approximate this solution, where θ represents the network's parameters. The total loss function is:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}}(\theta) + \lambda_{\text{ic}}\mathcal{L}_{\text{ic}}(\theta) \tag{1.1}$$

where:

- $\mathcal{L}_{\text{data}}$ is the **data loss**: measures the discrepancy between network predictions and observed data

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} \left(u_\theta(x_i,t_i) - u_i^{\text{obs}}\right)^2 \tag{1.2}$$

- $\mathcal{L}_{\text{phys}}$ is the **physics loss**: enforces the differential equation at randomly chosen points, called collocation points, in a certain region.

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_{\text{col}}} \sum_{j=1}^{N_{\text{col}}} \left(\mathcal{F}\left(u_\theta, \frac{\partial u_\theta}{\partial x}, \frac{\partial u_\theta}{\partial t}, \dots\right)\Big|_{x=x_j,t=t_j}\right)^2 \tag{1.3}$$

  where $\mathcal{F}$ represents the differential equation operator: the solution u has $\mathcal{F}(u(x,t))=0$ for $\forall x,t$.

- $\mathcal{L}_{\text{ic}}$ is the **initial/boundary condition loss**: enforces initial and boundary conditions

$$\mathcal{L}_{\text{ic}} = \frac{1}{N_{\text{ic}}} \sum_{k=1}^{N_{\text{ic}}} \left(u_\theta(x_k,t_0) - u_0(x_k)\right)^2 \tag{1.4}$$

- λ_{phys} and λ_{ic} are **loss weights** controlling the trade-off between fitting data and satisfying physics.

1.1.1 Advantages and disadvantages of PINNs

Advantages

+ **No mesh required**: Unlike finite difference or finite element methods, PINNs do not require discretization of the spatial and temporal domains; collocation points.

+ **Inverse problems**: Can be used to identify unknown physical parameters from data.

+ **Extendability**: The neural network learns about the workings of the physical system; this allows it to make meaningful predictions in regions that it was not trained for

1.1.2 Implementation Details

In practice, a PINN is typically implemented as follows:

1. **Network architecture**: A fully-connected feedforward neural network with several hidden layers and activation functions (e.g., tanh, GELU).
2. **Automatic differentiation**: Using frameworks like PyTorch or TensorFlow to compute any order derivatives.
3. **Optimization**: Train the network using gradient-based optimizers (e.g., Adam, L-BFGS) to minimize the combined loss function.
4. **Collocation points**: Sample interior collocation points to evaluate the physics residual and boundary/initial condition points separately.

1.2 The Damped Harmonic Oscillator

1.2.1 Problem Description

The damped harmonic oscillator is one of the most fundamental systems in classical mechanics, describing the motion of a mass attached to a spring with damping. It is governed by the following second-order ordinary differential equation:

$$m\ddot{y}(t) + c\dot{y}(t) + ky(t) = 0 \tag{1.5}$$

where:

- m is the mass [kg]
- c is the damping coefficient [kg/s]
- k is the spring stiffness [N/m]
- $y(t)$ is the displacement [m]
- $\dot{y}(t) = \frac{dy}{dt}$ is the velocity
- $\ddot{y}(t) = \frac{d^2y}{dt^2}$ is the acceleration

The system is typically subject to initial conditions:

$$y(0) = y_0, \quad \dot{y}(0) = v_0 \tag{1.6}$$

1.2.2 Physical Characterization

The behavior of the damped harmonic oscillator can be characterized using two non-dimensional quantities:

Definition 1.2.1 (Natural Frequency)**.** The **undamped natural frequency** is given by:

$$\omega_0 = \sqrt{\frac{k}{m}} \tag{1.7}$$

Definition 1.2.2 (Damping Ratio)**.** The **damping ratio** is given by:

$$\zeta = \frac{c}{2\sqrt{mk}} = \frac{c}{2m\omega_0} \tag{1.8}$$

The damping ratio determines the system's behavior:

- **Underdamped** ($\zeta < 1$): Oscillatory response with exponential decay
- **Critically damped** ($\zeta = 1$): Fastest non-oscillatory return to equilibrium
- **Overdamped** ($\zeta > 1$): Slow non-oscillatory return to equilibrium

1.2.3 Analytical Solution

For the underdamped case ($\zeta < 1$), the analytical solution is:

$$y(t) = e^{-\zeta\omega_0 t}\left(A\cos(\omega_d t) + B\sin(\omega_d t)\right) \tag{1.9}$$

where $\omega_d = \omega_0\sqrt{1-\zeta^2}$ is the **damped natural frequency**, and constants A, B are determined by initial conditions:

$$A = y_0 \tag{1.10}$$

$$B = \frac{v_0 + \zeta\omega_0 y_0}{\omega_d} \tag{1.11}$$

1.2.4 Application to PINNs

The PINN approach for the damped oscillator involves:

1. **Data generation**: Generate synthetic noisy observations by solving the ODE and adding measurement noise.
2. **Forward problem**: Train a PINN to predict $y(t)$ given the system parameters (m, c, k).
3. **Inverse problem**: Train a PINN to identify unknown parameters (ω_0, ζ) from noisy displacement measurements.

The physics loss enforces:

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_{\text{col}}}\sum_{j=1}^{N_{\text{col}}}\left(m\ddot{u}_\theta(t_j) + c\dot{u}_\theta(t_j) + ku_\theta(t_j)\right)^2 \tag{1.12}$$

where derivatives $\dot{u}_\theta$ and $\ddot{u}_\theta$ are computed via automatic differentiation.

1.3 Burgers' Equation

1.3.1 Problem Description

Burgers' equation is a fundamental nonlinear partial differential equation that arises in fluid dynamics, gas dynamics, and heat conduction. It combines nonlinear advection with linear diffusion:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2} \tag{1.13}$$

where:

- $u(x,t)$ is the solution variable (velocity in fluid dynamics)
- $\nu > 0$ is the viscosity coefficient [m^2/s]
- x is the spatial coordinate [-7, 7] (typical domain)
- t is the temporal coordinate [0, 3] (typical domain)

The equation is supplemented with:

1. **Initial condition**: $u(x,0) = u_0(x)$ for $x \in [x_{\min}, x_{\max}]$
2. **Boundary conditions**: Typically Dirichlet: $u(x_{\min}, t) = u(x_{\max}, t) = 0$

1.3.2 Physical Interpretation

The three terms in Burgers' equation have clear physical meanings:

- $\frac{\partial u}{\partial t}$: **Time derivative** — rate of change of the solution
- $u\frac{\partial u}{\partial x}$: **Advection** — nonlinear transport of the solution field (sharpening effects)
- $\nu\frac{\partial^2 u}{\partial x^2}$: **Diffusion** — smoothing due to viscosity (dissipation)

The balance between advection (nonlinear steepening) and diffusion (smoothing) creates complex dynamics:

- Small ν: Advection dominates, steep fronts form and can develop shocks
- Large ν: Diffusion dominates, smooth solutions evolve slowly
- Moderate ν: Intricate interplay creates rich dynamics

1.3.3 Cole-Hopf Transformation

A remarkable property of Burgers' equation is that it can be linearized via the Cole-Hopf transformation:

$$u = -2\nu \frac{\frac{\partial \phi}{\partial x}}{\phi} \tag{1.14}$$

This transforms Burgers' equation into the linear heat equation:

$$\frac{\partial \phi}{\partial t} = \nu \frac{\partial^2 \phi}{\partial x^2} \tag{1.15}$$

which has known analytical solutions. However, for PINN applications, we work with the nonlinear form directly.

1.3.4 Benchmark Initial Conditions

Two standard initial conditions are commonly used:

Gaussian Pulse

$$u(x, 0) = \exp\left(-\frac{x^2}{2}\right) \tag{1.16}$$

This represents a smooth, localized disturbance that diffuses over time.

N-Wave

$$u(x, 0) = \exp\left(-\frac{(x-1)^2}{2}\right) - \exp\left(-\frac{(x+1)^2}{2}\right) \tag{1.17}$$

This represents a wave-like structure with positive and negative lobes that interact via nonlinear advection.

1.3.5 Challenges for Numerical Methods

Burgers' equation presents several challenges:

- **Nonlinearity**: Standard linear solvers do not apply. Solutions can develop steep gradients and shocks.
- **Multiscale behavior**: Solutions exhibit both rapid changes (shocks) and smooth regions.
- **Conservation laws**: Proper numerical schemes must preserve physical conservation properties.
- **Stability**: Explicit time-stepping schemes require stringent CFL conditions.

1.3.6 Application to PINNs

PINNs offer a meshfree alternative for solving Burgers' equation:

1. **Network input**: (x, t) pairs representing spatial-temporal locations
2. **Network output**: $u(x, t)$ the solution at that location
3. **Physics residual**: Computed via automatic differentiation:

$$r(x,t) = \frac{\partial u_\theta}{\partial t} + u_\theta \frac{\partial u_\theta}{\partial x} - \nu \frac{\partial^2 u_\theta}{\partial x^2} \tag{1.18}$$

4. **Loss function**: Combines data loss, physics residual penalty, and initial/boundary condition enforcement

1.3.7 Solution Behavior

The solution behavior depends critically on the viscosity parameter ν:

- **Low viscosity** ($\nu \to 0$): Solutions approach those of the inviscid Burgers equation (purely hyperbolic), which can develop shocks
- **High viscosity**: Solutions become smooth and decay toward zero
- **Moderate viscosity**: Rich transient dynamics with traveling wave formation

Chapter 2

Understanding a Given Solution